LAB # 7

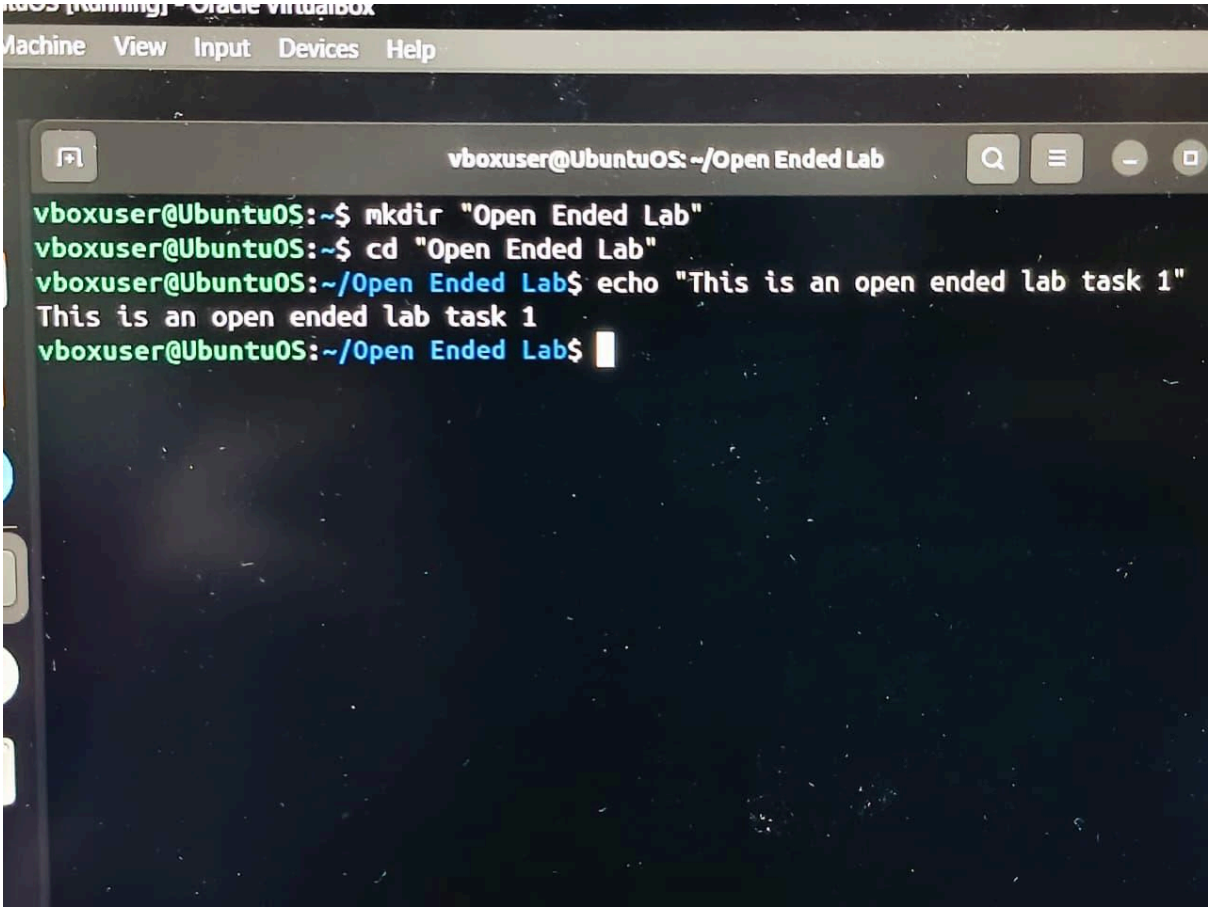
Open Ended Lab

Objective :

To simulate CPU Scheduling Algorithms. (Priority, Round Robin)

Task 1:

Create a folder called Open Ended Lab And Create a file with text “This is an open ended lab “



Task 2 :

Write a single C program that fetches the current process's unique ID (PID). The program should then simulate a state transition by moving from the Running State to the Waiting State for a brief period, and finally back to the Running State to finish execution. This demonstrates how operating systems track and transition processes.

Source Code :

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("State: RUNNING\n");
    printf("Current Process ID (PID): %d\n", getpid());

    printf("State: WAITING (Simulating I/O or sleep for 2 seconds...)\n");
    sleep(2);

    printf("State: RUNNING (Resumed execution)\n");
    printf("Process completed successfully.\n");

    return 0;
}
```

Output:

```
State: RUNNING
Current Process ID (PID): 1073385
State: WAITING (Simulating I/O or sleep for 2 seconds...)
State: RUNNING (Resumed execution)
Process completed successfully.
```

Task 3 :

Source Code :

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* squareMatrix(void* arg) {
    long thread_id = (long)arg;

    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    printf("Thread %ld starting calculation...\n", thread_id);
    printf("Result Matrix for Thread %ld:\n", thread_id);

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            matrix[i][j] = matrix[i][j] * matrix[i][j];
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
    printf("-----------------------------------\n");

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[3];

    for (long i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, squareMatrix, (void*)i);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads finished execution.\n");
    return 0;
}
```

Output :

```
Thread 2 starting calculation...
Result Matrix for Thread 2:
1       4       9
16      25      36
49      64      81
-----------------------------------
Thread 1 starting calculation...
Result Matrix for Thread 1:
1       4       9
16      25      36
49      64      81
-----------------------------------
Thread 0 starting calculation...
Result Matrix for Thread 0:
1       4       9
16      25      36
49      64      81
-----------------------------------
All threads finished execution.
```

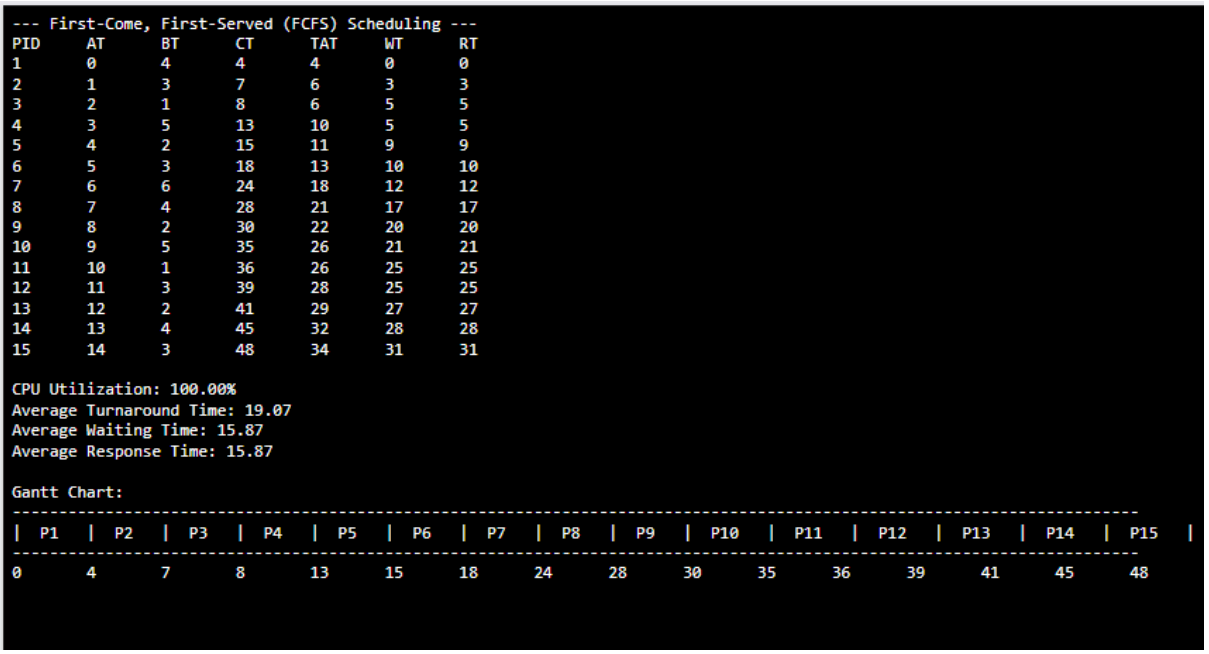
Task 4 :

Task: Write a C program that calculates CPU utilization, average waiting time, turnaround time, and response time using the First-Come, First-Served (FCFS) scheduling algorithm. The program must handle at least 15 CPU burst times with hardcoded arrival times. Display a text-based Gantt chart on the screen.

Source Code :

```c
#include <stdio.h>
struct Process {
    int id, at, bt, ct, tat, wt, rt;
};
struct GanttSegment {
    int id, start, end;
};
void printGantt(struct GanttSegment chart[], int k) {
    printf("\nGantt Chart:\n");
    for (int i = 0; i < k; i++) {
        printf("--------");
    }
    printf("-\n|");
    for (int i = 0; i < k; i++) {
        if (chart[i].id == -1)
            printf(" IDLE  |");
        else
            printf("  P%d   |", chart[i].id);
    }
    printf("\n");
    for (int i = 0; i < k; i++) {
        printf("--------");
    }
    printf("-\n");
    printf("%d", chart[0].start);
    for (int i = 0; i < k; i++) {
        printf("\t%d", chart[i].end);
    }
    printf("\n");
}

int main() {
    int n = 15;
    struct Process p[15] = {
        {1, 0, 4}, {2, 1, 3}, {3, 2, 1}, {4, 3, 5}, {5, 4, 2},
        {6, 5, 3}, {7, 6, 6}, {8, 7, 4}, {9, 8, 2}, {10, 9, 5},
        {11, 10, 1}, {12, 11, 3}, {13, 12, 2}, {14, 13, 4}, {15, 14, 3}
    };

    struct GanttSegment chart[30];
    int k = 0, t = 0, total_bt = 0;
    float total_tat = 0, total_wt = 0, total_rt = 0;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].at > p[j + 1].at) {
                struct Process temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        total_bt += p[i].bt;
        if (t < p[i].at) {
            chart[k].start = t;
            chart[k].id = -1;
            t = p[i].at;
            chart[k].end = t;
            k++;
        }

        p[i].rt = t - p[i].at;
        chart[k].start = t;
        chart[k].id = p[i].id;
        t += p[i].bt;
        chart[k].end = t;
        k++;

        p[i].ct = t;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        total_tat += p[i].tat;
        total_wt += p[i].wt;
        total_rt += p[i].rt;
    }
    float cpu_utilization = ((float)total_bt / t) * 100;
    printf("\n--- First-Come, First-Served (FCFS) Scheduling ---\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }
    printf("\nCPU Utilization: %.2f%%\n", cpu_utilization);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);
    printf("Average Response Time: %.2f\n", total_rt / n);

    printGantt(chart, k);

    return 0;
}
```

Output :



Task 5: Write a C program that takes hardcoded process details including process ID, arrival time, burst time, and priority configuration. Calculate the average waiting time and turnaround time using Non-Preemptive Priority Scheduling. Display a text-based Gantt chart on the screen.

Source Code : Output

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
struct Process {
    int id, at, bt, pr, ct, tat, wt, is_completed;
};
struct GanttSegment {
    int id, start, end;
};
void printGantt(struct GanttSegment chart[], int k) {
    printf("\nGantt Chart:\n");
    for (int i = 0; i < k; i++) {
        printf("--------");
    }
    printf("-\n|");
    for (int i = 0; i < k; i++) {
        if (chart[i].id == -1)
            printf(" IDLE  |");
        else
            printf("  P%d   |", chart[i].id);
    }
    printf("\n");
    for (int i = 0; i < k; i++) {
        printf("--------");
    }
    printf("-\n");
    printf("%d", chart[0].start);
    for (int i = 0; i < k; i++) {
        printf("\t%d", chart[i].end);
    }
    printf("\n");
}
int main() {
    int n = 5;
    struct Process p[5] = {
        {1, 0, 4, 2, 0, 0, 0, 0},
        {2, 1, 3, 3, 0, 0, 0, 0},
        {3, 2, 1, 4, 0, 0, 0, 0},
        {4, 3, 5, 5, 0, 0, 0, 0},
        {5, 4, 2, 1, 0, 0, 0, 0}
    };
    int t = 0, completed = 0;
    float total_wt = 0, total_tat = 0;
    struct GanttSegment chart[100];
    int k = 0;

    while (completed != n) {
        int idx = -1;
        int min_pr = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= t && p[i].is_completed == 0) {
                if (p[i].pr < min_pr) {
                    min_pr = p[i].pr;
                    idx = i;
                } else if (p[i].pr == min_pr) {
                    if (p[i].at < p[idx].at) {
                        idx = i;}}}}
        if (idx != -1) {
            chart[k].start = t;
            chart[k].id = p[idx].id;
            t += p[idx].bt;
            chart[k].end = t;
            k++;

            p[idx].ct = t;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            p[idx].is_completed = 1;
            completed++;
        } else {
            if (k > 0 && chart[k - 1].id == -1) {
                chart[k - 1].end = t + 1;
            } else {
                chart[k].start = t;
                chart[k].id = -1;
                chart[k].end = t + 1;
                k++;
            }
            t++;
        }
    }

    printf("\n--- Non-Preemptive Priority Scheduling ---\n");
    printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);
    printGantt(chart, k);
    return 0;
}
```

```
--- Non-Preemptive Priority Scheduling ---
PID     AT      BT      PR      CT      TAT     WT
1       0       4       2       4       4       0
2       1       3       3       9       8       5
3       2       1       4       10      8       7
4       3       5       5       15      12      7
5       4       2       1       6       2       0
Average Turnaround Time: 6.80
Average Waiting Time: 3.80

Gantt Chart:
----------------------------------------
|  P1   |  P5   |  P2   |  P3   |  P4   |
----------------------------------------
0       4       6       9       10      15
```